

CHE655: LAB 3

NAME: AADITYAAMLAN PANDA

ROLL: 220007

INTRODUCTION

In the lab, we were expected to find the Singular Value Decomposition of a matrix using repetitive QR decomposition and obtain V, sigma and U

METHODOLOGY

Step 1: First the eigenvalues of $A^T A$ are calculated.

Step 2: This is done via $A^T A = QR$, then R^*Q results in next step is further decomposed. This is repeated to 1000 repetitions. The final R^*Q approximately gives eigenvalues at the diagonal.

Step 3: Qs in all steps are accumulated and multiplied which results in matrix of eigenvectors which is equal to the matrix V.

Step 4: The same steps are done with AA^T in which the eigenvector matrix equals U

Step 5: Sigma is a diagonal matrix whose diagonal elements are square roots of the eigenvalues of $A^T A$ or the AA^T matrix

RESULTS

INPUT: -

[[3. 0.]

[4. 5.]]

Printing U after 99th repetitions:-

[[0.82165886 -0.56997957]

[0.56997957 0.82165886]]

Printing sigma after 99th repetitions:-

[[6.70820393 0.]

[0. 2.23606798]]

Printing V after 99th repetitions:-

[[0.97824524 -0.20745182]

[0.20745182 0.97824524]]

Column norm of U

1.0

Column norm of V

1.0000000000000002

As one can see above the results of sigma matches with obtain in the class example $\sqrt{45}$ and $\sqrt{5}$. Also the U AND V are orthonormal with norms = 1

CODE

```
# -*- coding: utf-8 -*-  
"""
```

Spyder Editor

This is a temporary script file.
"""

```
import numpy as np  
import math
```

```
def find_col_norm(V,pos,n):  
    s = 0  
    for i in range(n):  
        s+=V[i][pos]*V[i][pos]  
  
    s = math.sqrt(s)  
    return s
```

```
def find_dot(A,Q,apos,qpos,n):  
    s = 0  
    for i in range(n):  
        s+=A[i][apos]*Q[i][qpos]  
  
    return s
```

```
def find_vec_norm(V,n):  
    s = 0  
    for i in range(n):  
        s+=V[i]*V[i]  
  
    s = math.sqrt(s)  
    return s
```

```
def find_transpose(A,n,m):  
    Atrp = np.zeros((m,n))  
    for i in range(n):  
        for j in range(m):  
            Atrp[j][i] = A[i][j]  
    return Atrp
```

```
def matrix_multiply(A1,A2,n,t,m):  
    Ares = np.zeros((n,m))  
    for i in range(n):  
        for j in range(m):  
            for k in range(t):  
                Ares[i][j]+=(A1[i][k]*A2[k][j])  
    return Ares
```

```
def extract_col(A,n,j):  
    col_vec = np.zeros((n,1))  
    for i in range(n):  
        col_vec[i] = A[i][j]  
    return col_vec
```

```
print('Enter the number of rows = number of columns and then enter the elements at every position')
```

```

n = int(input('Enter the number of rows:'))
m = int(input('Enter the number of columns:'))

A = np.zeros((n,m))
for i in range(n):
    for j in range(m):
        A[i][j] = float(input("Enter element at position "+str(i)+" "+str(j)+" :"))

N = m
Q = np.zeros((N,N))
R = np.zeros((N,N))

Atrp = find_transpose(A,n,m)
print(Atrp)
A_curr = matrix_multiply(Atrp,A,m,n,m)

print(A_curr)
Q_final = np.zeros((N,N))

for rep in range(100):
    # Finding q1
    norm_a1 = find_col_norm(A_curr,0,N)
    for i in range(N):
        Q[i][0] = A_curr[i][0]/norm_a1

#defining dp for overlapping subproblems....row stands for A and column stands for Q
dp = np.zeros((N,N))
vis = np.zeros((N,N))

# Finding qr
for j in range(N):
    if j==0:
        continue
    perp = np.zeros(N)
    for i in range(N):
        s = 0

        for r in range(j):

            if vis[j][r] == 0:
                dp[j][r] = find_dot(A_curr,Q,j,r,N)
                vis[j][r] = 1

            s+=dp[j][r]*Q[i][r]

        perp[i] = A_curr[i][j]-s

    perp_norm = find_vec_norm(perp,N)

    for i in range(N):
        Q[i][j] = perp[i]/perp_norm

#print(Q)

```

```

if rep==0:
    Q_final = Q
else:
    temp = matrix_multiply(Q_final,Q,N,N,N)
    Q_final = temp

#Finding R:  $R_{ij} = Q_i \cdot A_j$ 
for i in range(N):
    for j in range(N):
        if i<=j:
            if vis[j][i]==0:
                dp[j][i] = find_dot(A_curr,Q,j,i,N)
                vis[j][i] = 1
            R[i][j] = dp[j][i]
#print(R)
A_curr = matrix_multiply(R, Q, N, N, N)

print("Printing A after "+str(rep)+"th repetitions:-")
print(A_curr)

print("Printing Q after "+str(rep)+"th repetitions:-")
print(Q_final)

V = Q_final

U = np.zeros((n,n))

sigma = np.zeros((N,N))
for i in range(N):
    sigma[i][i] = math.sqrt(A_curr[i][i])

A_curr = matrix_multiply(A,Atrp,m,n,m)

print(A_curr)
Q_final = np.zeros((N,N))

for rep in range(100):
    # Finding q1
    norm_a1 = find_col_norm(A_curr,0,N)
    for i in range(N):
        Q[i][0] = A_curr[i][0]/norm_a1

#defining dp for overlapping subproblems....row stands for A and column stands for Q
dp = np.zeros((N,N))
vis = np.zeros((N,N))

# Finding qr
for j in range(N):
    if j==0:
        continue
    perp = np.zeros(N)
    for i in range(N):
        s = 0

        for r in range(j):

```

```

        if vis[j][r] == 0:
            dp[j][r] = find_dot(A_curr, Q, j, r, N)
            vis[j][r] = 1

        s += dp[j][r] * Q[i][r]

    perp[i] = A_curr[i][j] - s

perp_norm = find_vec_norm(perp, N)

for i in range(N):
    Q[i][j] = perp[i] / perp_norm

#print(Q)

if rep == 0:
    Q_final = Q
else:
    temp = matrix_multiply(Q_final, Q, N, N, N)
    Q_final = temp

#Finding R:  $R_{ij} = Q_i * A_j$ 
for i in range(N):
    for j in range(N):
        if i <= j:
            if vis[j][i] == 0:
                dp[j][i] = find_dot(A_curr, Q, j, i, N)
                vis[j][i] = 1
            R[i][j] = dp[j][i]
#print(R)
A_curr = matrix_multiply(R, Q, N, N, N)

print("Printing A after " + str(rep) + "th repetitions:-")
print(A_curr)

print("Printing Q after " + str(rep) + "th repetitions:-")
print(Q_final)

U = Q_final

print("Printing U after " + str(rep) + "th repetitions:-")
print(U)

print("Printing sigma after " + str(rep) + "th repetitions:-")
print(sigma)

print("Printing V after " + str(rep) + "th repetitions:-")
print(V)

print("Column norm of U")
nm = find_col_norm(U, 0, N)
print(nm)

print("Column norm of V")

```

```
nm_ = find_col_norm(V,0,N)
print(nm_)
```